SVV Structures Assignment 2025 – 2026

AE3212-II Simulation, Verification and Validation course
Written by dr. ir. J.M.J.F. van Campen
Updated by dr. Vahid Yaghoubi
February 2026

Background

The design of spacecraft structures is an engineering challenge. Not only are spacecraft structures subjected to an environment that is difficult to simulate on earth, but a spacecraft structure must perform as predicted within small tolerances. The recently launched James Webb telescope provides an example. The thermal deformation of the composite structure supporting the mirrors (fig. 1) is allowed to contribute up to a few nanometers to the so-called Wave Front Error of the telescope infrared observations [1]. For general spacecraft structure, further complications are extreme structural requirements in terms of lightweight and foldable design that must withstand harsh vibrational loads during launch, and significant thermal loads once in orbit. Proper understanding of simulation, verification and validation (SVV) is therefore paramount for economically viable spaceflight.

The above is the background for the AE3212-II SVV structures assignment in front of you. For the purpose of this assignment, we pretend that you are a group of engineers running your own company in spacecraft structures, which is a subcontractor for Spacecraft structures Project (SpacePro). SpacePro is developing a new satellite. Your company is asked to design the structure of an antenna dish. The antenna dish will be connected to the spacecraft which is designed by SpacePro. Note, that the structures, loads and dimensions used in this assignment are fictional. They have been chosen to simplify the complexity of spacecraft structures to a level where the simulation, verification and validation of a structural design tool come into focus, rather than the complex structure itself.

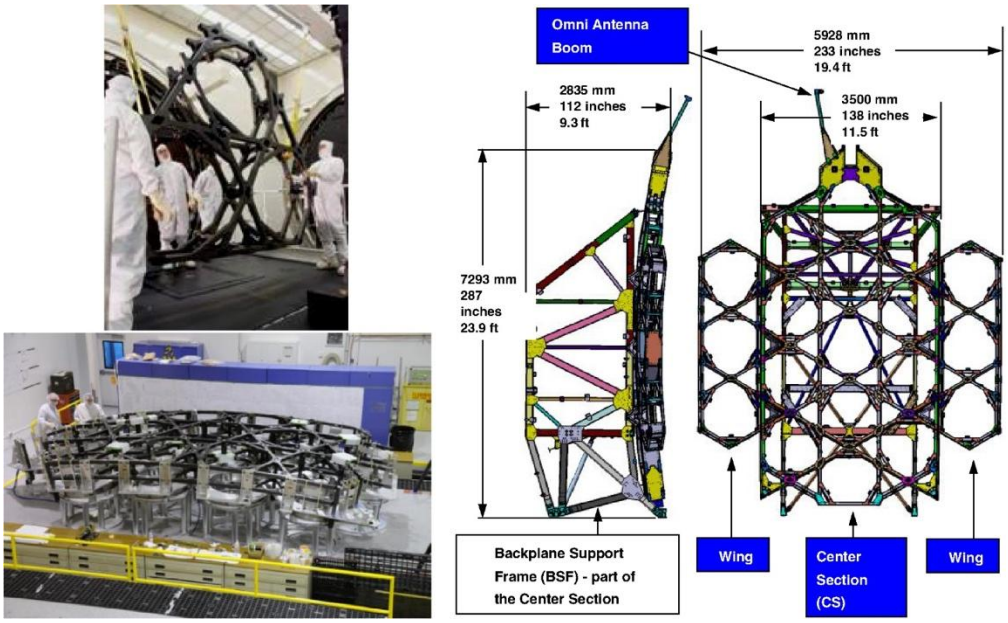


Figure 1: right: backplane assembly supporting the primary mirror; left: 1/6th scale structure used for testing [1].

Figure 2: SpacePro logo

Content

BACKGROUND **1**

CONTENT **2**

1. INTRODUCTION **3**

2. SPECIFICATIONS OF THE ANTENNA DISH STRUCTURE **4**

3. SPECIFICATIONS MECHRES2D **5**

ELEMENT TYPES 6
Euler-Bernoulli Straight Beam *6*
Rod Element *8*
LOADING VECTORS 9
ANALYSIS 10
ELEMENT STRAINS 11
ELEMENT STRESSES 12
CODE STRUCTURE 12
0. User Input *12*
1. Input Check *13*
2. Creation of Parts *13*
3. Creation of Elements *13*
4. Assembling the Structure *13*
5. Application of Loads and Boundary Conditions *13*
6. Analysis *13*
7. Plotting *13*
8. Programme Output *13*

4. AVAILABLE EXPERIMENTAL DATA **14**

BEAM STRUCTURE 14
TRUSS STRUCTURE 15

5. STRUCTURES ASSIGNMENT **17**

VERIFICATION AND VALIDATION PLAN 18
STRUCTURES REPORT 18

REFERENCES **19**

APPENDIX A: MECHRES2D AND CONSTGEOM PYTHON SCRIPT **20**

1. Introduction

SpacePro wants your team to complete a new Python code of a preliminary design tool and to perform the necessary verification and validation tests.

The preliminary design tool is called MechRes2D, which is short for Mechanical Response 2D. The design tool simulates the mechanical response of a 2D cross-section of the structure of an antenna dish.

MechRes2D can simulate two types of structures:

- Truss structures
- Beam structures

To understand the difference between beam and truss elements, it will be helpful to revisit the first year Statics Course, AE1130-I. In chapter 6 of "Engineering Mechanics Statics " R.C. Hibbeler, 14th SI edition, trusses, frames and mechanisms are explained. The elements in a truss structure are assumed only to be able to carry axial loads, but no bending moments. In contrast, beam elements can carry bending moments and axial loads. In the statics course the two types of elements are then combined in what is called a frame. It will be possible to analyze the structures given in this course using your knowledge of the first-year statics course.

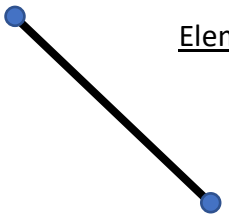
The shape of the antenna dish influences its performance. Therefore, deformations must remain within prescribed bounds. The code takes as input the geometrical design of the structure and materials selected from a user database. The code gives the following outputs:

- Nodal displacements
- Element axial stresses
- Element bending moment (if applicable)
- First 5 eigen frequencies of the structure

For validation, the test results of a simple mechanical experiment are available.

Terminology

<u>Node</u>: Defined point in space
In 2D, a node has 3 degrees of freedom (DOF):
- Displacement in x direction
- Displacement in y-direction
- Rotation about the z-axis



<u>Element</u>: Truss and beam elements are defined by:
- 2 nodes
- Section properties (A,I)
- Material properties (E,ρ)

2. Specifications of the Antenna Dish Structure

There are two main types of structure for which SpacePro uses MechRes2D, (fig. 3). For the beam type structure (fig. 3.a), only the dish itself (circular part) is subjected to an equal thermal load, whereas the flexures (indicated in green) it is resting on are not. For the truss type structure (fig 3.b) the entire structure is subjected to an equal thermal load.

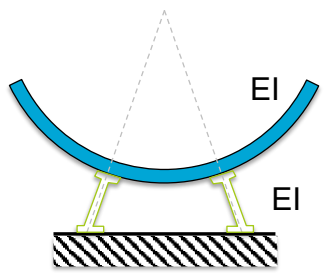


Figure 3.a: Beam type

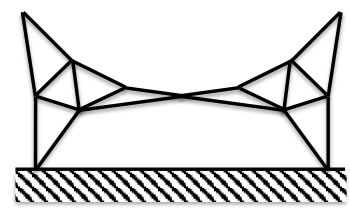


Figure 3.b: Truss Type

Figure 3: Two types of structure for which MechRes2D is used mainly

There is a fixed parametrization scheme (fig. 4). for the two types of structures discussed above. Typical parameter sets used for both types of structure are given in Table 1.

N.B: A structure using beam elements can be discretized in more detail by dividing one beam element into multiple beam elements. The discretization of a truss structure cannot be increased. The reason for this difference is the nature of the elements. Dividing a truss element into multiple linked truss elements will result in an ill-defined system of equations, because the nodes that are added will only be constrained in the direction of the truss element, but not in the direction perpendicular to it. There is a possible work around: replace a truss element by a beam element whilst taking care that no moments will be applied to the beam element at the nodes that define the original truss element. It is recommended that you only attempt this, in case your project group has time to spare.

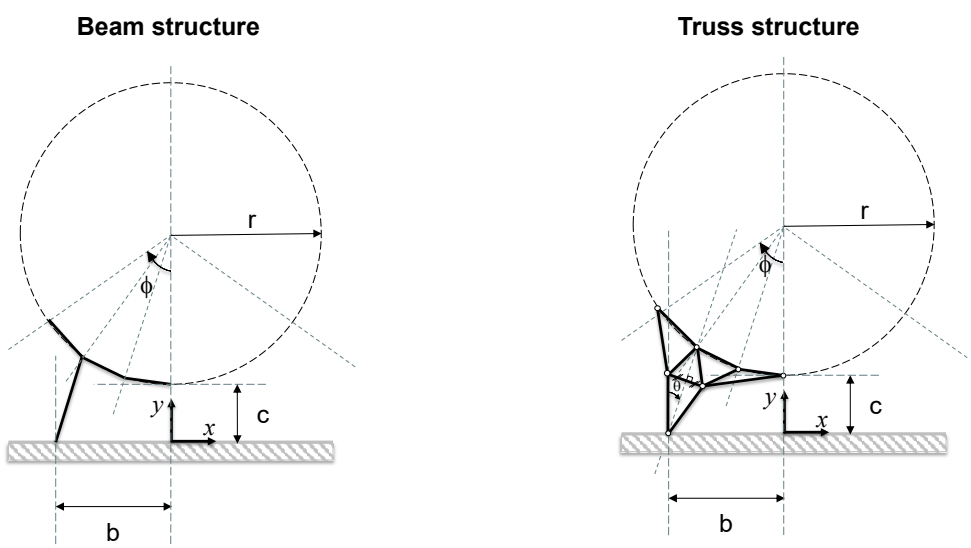


Figure 4: Discretization of the two types of antenna dish structure

Table 1: typical parameter values for antenna dish structure

	Set 1	Set 2	Set 3
b [mm]	700	850	350
c [mm]	600	300	350
r [mm]	2400	5000	1100
φ [°]	30	15	20
Material[1]	1	2	3

3. Specifications MechRes2D

Most of the theory used in MechRes2D can be found in Aircraft Structures for Engineering Students by Megson, fifth or sixth edition [2]. Especially chapter 6 is relevant for the assignment. Knowledge of the AE2135-II Vibrations course will be useful too. In this section the element types and the solution procedures that have been implemented in MechRes2D will be described.

Please note, that next to Megson, the assignment draws on knowledge from multiple courses, as can be seen from the overview below.

AE1130-I Engineering Mechanics: Statics:

- Analysis of truss structures
- Internal loading of beam elements

AE1108-II Aerospace Mechanics of Materials

- Stress and strain relations

WI1403LR Linear Algebra

- Matrix operations
- Coordinate transformations
- Eigenvalue analysis

AE2135-I Structural Analysis & Design

- Energy methods

AE2135-II Vibrations

- Modal analysis

WI2180LR-II Probability and Statistics

- Statistical procedures to analyze data

Please note that, in this document all matrices will be indicated with square brackets and all vectors will be indicated with curly brackets. Furthermore, please note, that the equations for the beam element in this document have been taken from Appendix A of Gan [3]. The description of the matrix method in Megson [2], is more than sufficient to use these equations.

[1] The material database that is referred to in Table 1 is hardcoded in MechRes2D.

Element Types

The user of MechRes2D can choose between two element types: beam elements or rod elements, as is explained on page 3 of this document.

Euler-Bernoulli Straight Beam

The following equation is used to describe the relationship between the deflection of an Euler-Bernoulli beam and the load applied to it:

$$\frac{d^2}{dx^2}\left(EI\frac{d^2w}{dx^2}\right) = q$$

To make the solution of this differential equation tractable for a non-trivial geometry, like for example an antenna structure, a structure can be divided into a large number of geometrically simple elements for which the differential equation is easy to solve. This leads to the Euler-Bernoulli straight beam. A graphical description of the Euler-Bernoulli straight beam is given (fig. 5). It is a straight element connecting two nodes, i and j.

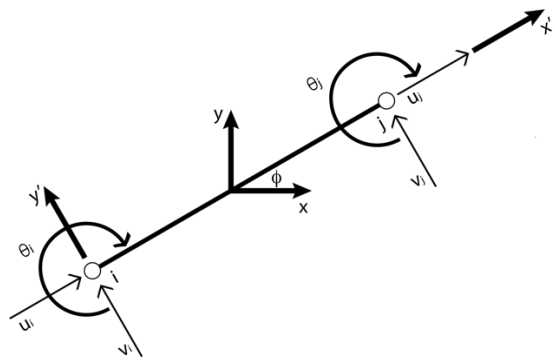


Figure 5: Euler-Bernoulli Straight Beam, modified from Megson [2]

Assumptions made in the description of the Euler-Bernoulli straight beam are the following:

- Plane sections of the cross-section of the beam remain plane
- The slopes of the deformed beams are small, i.e. the angles remain small

The stiffness matrix of the Euler-Bernoulli straight beam is given in equation 1. Where E, A, I and L refer to the properties of the element.

$$[\overline{K}_{ij}] = \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & 12EI/L^3 & 6EI/L^2 & 0 & -12EI/L^3 & 6EI/L^2 \\ 0 & 6EI/L^2 & 4EI/L & 0 & -6EI/L^2 & 2EI/L \\ -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -12EI/L^3 & -6EI/L^2 & 0 & 12EI/L^3 & -6EI/L^2 \\ 0 & 6EI/L^2 & 2EI/L & 0 & -6EI/L^2 & 4EI/L \end{bmatrix} \quad (1)$$

The above stiffness matrix is given in the local coordinate system of the element, x' and y'. Each node of the element has three degrees of freedom. Columns and rows 1 to 3 correspond to the

displacement in parallel to the x'-axis, the displacement parallel to the y'-axis, and the rotation about the z'-axis respectively of node i. Similarly, columns and rows 4 to 6 correspond to the displacement in parallel to the x'-axis, the displacement parallel to the y'-axis, and the rotation about the z'-axis respectively of node j.

The local stiffness matrix must be transformed to the global coordinate system. This way it will be possible to assemble a global stiffness matrix for the entire structure. To transform the local stiffness matrix it must be rotated to the global coordinate system of the structure. (For more information on matrix rotations, please review the first-year linear algebra course.)

The rotation matrix is given in equation 2. Where $\lambda = \cos\phi$ and $\mu = \sin\phi$. Angle ϕ is the rotation angle of the element w.r.t the x-axis of the global coordinate system. This angle should not be confused with the rotational degree of freedom θ_i (fig. 5).

$$[T] = \begin{bmatrix} \lambda & \mu & 0 & 0 & 0 & 0 \\ -\mu & \lambda & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \lambda & \mu & 0 \\ 0 & 0 & 0 & -\mu & \lambda & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \tag{2}$$

To give an example, let's assume the following rotation: $\phi = 30^\circ$. Then the rotation matrix becomes:

$$[T] = \begin{bmatrix} \frac{1}{2}\sqrt{3} & \frac{1}{2} & 0 & 0 & 0 & 0 \\ -\frac{1}{2} & \frac{1}{2}\sqrt{3} & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{2}\sqrt{3} & \frac{1}{2} & 0 \\ 0 & 0 & 0 & -\frac{1}{2} & \frac{1}{2}\sqrt{3} & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The contribution to the global stiffness matrix of the element follows from the transformation described in equation 3.

$$[K_{ij}] = [T]^{\mathsf{T}}[\bar{K}_{ij}][T] \tag{3}$$

The mass matrix of the beam element is given in equation 4.

$$[m] = [m_{\rho A}] + [m_{\rho I}] \tag{4}$$

Where:

$$[m_{\rho A}] = \frac{\rho A L}{420}\begin{bmatrix} 140 & 0 & 0 & 70 & 0 & 0 \\ 0 & 156 & 22L & 0 & 54 & -13L \\ 0 & 22L & 4L^2 & 0 & 13L & -3L^2 \\ 70 & 0 & 0 & 140 & 0 & 0 \\ 0 & 54 & 13L & 0 & 156 & -22L \\ 0 & -13L & -3L^2 & 0 & -22L & 4L^2 \end{bmatrix}$$

and

$$[m_{\rho I}] = \frac{\rho I}{30L}\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 36 & 3L & 0 & -36 & 3L \\ 0 & 3L & 4L^2 & 0 & -3L & -L^2 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -36 & -3L & 0 & 36 & -3L \\ 0 & 3L & -L^2 & 0 & -3L & 4L^2 \end{bmatrix}$$

Rod Element

A rod element as can be found in a pin-jointed framework or truss structure is shown (fig. 6). The element stiffness matrix of a rod in a pin-jointed framework is given in equation 5. The main assumption behind the rod element is that it can only transfer axial loads.

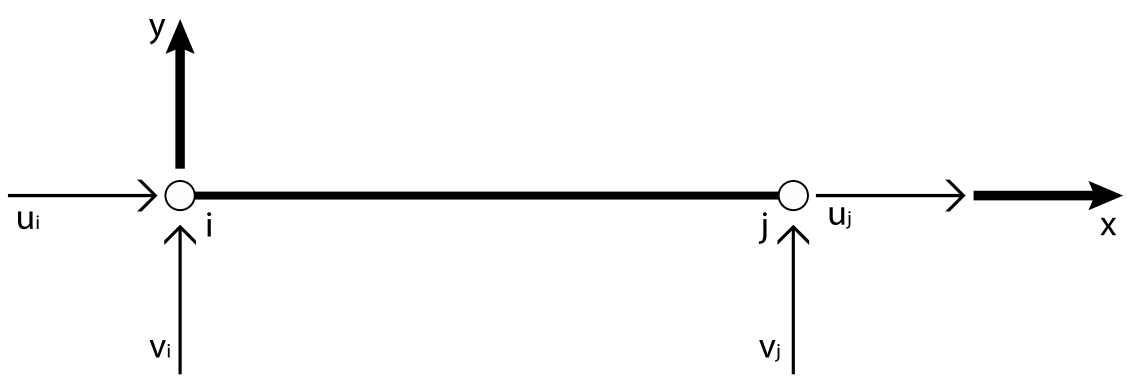


Figure 6: Rod element, modified from Megson [2] for $\phi = o^{\circ}$. The definition of ϕ is given in figure 5.

$$[\overline{K}_{ij}] = \frac{EA}{L}\begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{5}$$

The above stiffness matrix is given in the local coordinate system of the element, x' and y'. Each node of the element has two degrees of freedom. Columns and rows 1 to 2 correspond to the displacement in parallel to the x'-axis, the displacement parallel to the y'-axis respectively of node i. Similarly, columns and rows 3 to 4 correspond to the displacement in parallel to the x'-axis, the

displacement parallel to the y'-axis respectively of node j. Note that, contrary to the beam element, the truss element only has two degrees of freedom per node. The rotations at its nodes are not defined.

The rotation matrix for the rod element is given in equation 6. Where $\lambda = \cos\phi$ and $\mu = \sin\phi$. Angle ϕ is the rotation angle of the element w.r.t the x-axis of the global coordinate system, as defined in figure 5. The rotation itself is performed using equation 3, which results in the addition of the element to the global stiffness matrix of the structure.

$$[T] = \begin{bmatrix} \lambda & \mu & 0 & 0 \\ -\mu & \lambda & 0 & 0 \\ 0 & 0 & \lambda & \mu \\ 0 & 0 & -\mu & \lambda \end{bmatrix} \tag{6}$$

In order to be able to perform a modal analysis a mass matrix must be constructed. (Please review the second-year vibrations course for more information on modal analysis).

The mass matrix of the rod element is given in equation 7.

$$[m] = \frac{\rho AL}{6} \begin{bmatrix} 2 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \end{bmatrix} \tag{7}$$

Alternatively, the mass for rod elements can be lumped in the nodes. Effectively this means that the total mass of an element is equally divided over its two nodes. The lumped mass matrix is given in equation 8. Typically, it is used to reduce the computational effort required to evaluate the eigenfrequencies of the system, however this does not consider some mass inertia effects. Better results will be obtained by dividing a structure into more elements.

$$[\overline{m}] = \frac{\rho AL}{2} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{8}$$

Loading Vectors

The MechRes2D program allows for two types of load vectors, both of which can only be applied at the nodes of the model. Note that these vectors have a length of six. The first three entries correspond to the three degrees of freedom of node i of the element. The last three entries correspond to the three degrees of freedom of node j of the element.

First of all, the nodal loads due to thermal expansion, given in equation 9 for the Euler-Bernoulli beam element, where α is the thermal expansion coefficient. For the rod element the third entry for each of the two nodes is simply removed.

$$\{\vec{Q}\} = EA\alpha\Delta T \begin{Bmatrix} 1 \\ 0 \\ 0 \\ -1 \\ 0 \\ 0 \end{Bmatrix} \tag{9}$$

Next to the thermal load vector, nodal loads and moments can be applied as well, equation 10.

$$\{\vec{P}\} = \begin{Bmatrix} F_{x_i} \\ F_{y_i} \\ 0 \\ F_{x_j} \\ F_{y_j} \\ 0 \end{Bmatrix} \tag{10}$$

The global loading vectors can be assembled in the same way that the global stiffness matrix and mass matrix are assembled.

Analysis

MechRes2D performs a linear elastic analysis, described in equation 11. This equation expresses the static equilibrium of the system: the total sum of forces equals zero. (Please review the first-year statics course for more information on static equilibrium). Vector $\{\vec{P}\}$ contains all applied nodal loads, vector $\{\vec{Q}\}$ contains the thermal nodal loads, and vector $\{\vec{R}\}$ contains the nodal reaction forces, and vector $\{\vec{U}\}$ contains all nodal displacements.

$$[K]\{\vec{U}\} = \{\vec{P}\} - \{\vec{Q}\} + \{\vec{R}\} \tag{11}$$

Linear algebra can be used to rearrange the equilibrium equation in the following way:

$$\begin{bmatrix} K_r & K_{rs} \\ K_{sr} & K_s \end{bmatrix} \begin{Bmatrix} \overrightarrow{U_r} \\ \overrightarrow{U_s} \end{Bmatrix} = \begin{Bmatrix} \overrightarrow{P_r} \\ \overrightarrow{P_s} \end{Bmatrix} - \begin{Bmatrix} \overrightarrow{Q_r} \\ \overrightarrow{Q_s} \end{Bmatrix} + \begin{Bmatrix} \overrightarrow{R_r} \\ \overrightarrow{R_s} \end{Bmatrix} \tag{12}$$

Where r and s indicate the reduced and suppressed DOFs, $\{\overrightarrow{R_s}\}$ is the vector with all reaction forces. Reaction forces only exist where boundary conditions are specified, because of the third law of Newtown.
Equation 12 represents an important and powerful tool in mechanical analysis. It combines multiple simple concepts in a clever way.

In MechRes2D boundary conditions fully constrain a degree of freedom. As a result, $\{\overrightarrow{U_s}\} = \{\vec{0}\}$. The reduced displacement vector $\{\overrightarrow{U_r}\}$ contains the displacements of all nodes that have not been constrained.

N.B.: Equation 10 can only be solved if the structure is simply supported, but it will also work for statically indeterminate structures. In case the amount of boundary conditions is too small, some degrees can take any value. It will be useful to analyze a structure in advance by drawing a free-body-diagram and by analyzing its statical determinacy. Please review the statics course for more information.

The reaction forces and displacements are calculated by equations 13 and 14.

$$\{\overrightarrow{R_s}\} = [K_{sr}]\{\overrightarrow{U_r}\} - \left(\{\overrightarrow{P_s}\} - \{\overrightarrow{Q_s}\}\right) \tag{13}$$

$$\{\overrightarrow{U_r}\} = [K_r]^{-1}\left(\{\overrightarrow{P_r}\} - \{\overrightarrow{Q_r}\}\right) \tag{14}$$

To perform a modal analysis of the system, the equations of motion must be constructed. For this the stiffness matrix and the mass matrix of the structure must be combined as given in equation 15. (Please review the vibrations course for more information on the equation of motion). This equation gives the equations of motion for the constrained system.

$$[m]\left\{\ddot{\vec{U}}\right\} + [K]\{\vec{U}\} = \{\vec{P}\} - \{\vec{Q}\} + \{\vec{R}\} \tag{15}$$

The eigenfrequencies of the constrained system follow from solving the eigenvalue problem given in equation 16. (Please review the vibrations course for more information on how to compute eigenfrequencies).

$$\left|\frac{1}{\omega^2}[I_r] - [K_r]^{-1}[m_r]\right| = 0 \tag{16}$$

Where $[I_r]$ is an identity matrix with the size of the reduced system. MechRes2D will return the first 5 solutions.

Element Strains

Using the nodal displacement vector $\{\vec{U}\}$ the length of each element in deformed state can be calculated. Using equation 17 the strain of each element is calculated in MechRes2D, where x refers to the local coordinate system of the element. This equation is known as engineering strain.

$$\varepsilon_x = \frac{\Delta L}{L} \tag{17}$$

N.B: within the context of this course we are only considering finite elements that are defined by two nodes only. There are more complicated finite elements based on advanced integration schemes that use multiple nodes for one 2D element. These more advanced element types fall outside of the scope of this course.

Element Stresses

Using the stress strain relationship described in equation 18 the element stresses are calculated in MechRes2D, where x refers to the local coordinate system of the element. In case beam elements are being used the stresses that occur due to bending are ignored.

$$\sigma_x = E\varepsilon_x \tag{18}$$

Code Structure

The numbering used in the flow-diagram of MechRes2D (fig. 7) is identical to the numbering used in the comments of MechRes2D, see appendix A. Please use this numbering the navigate through MechRes2D more easily.

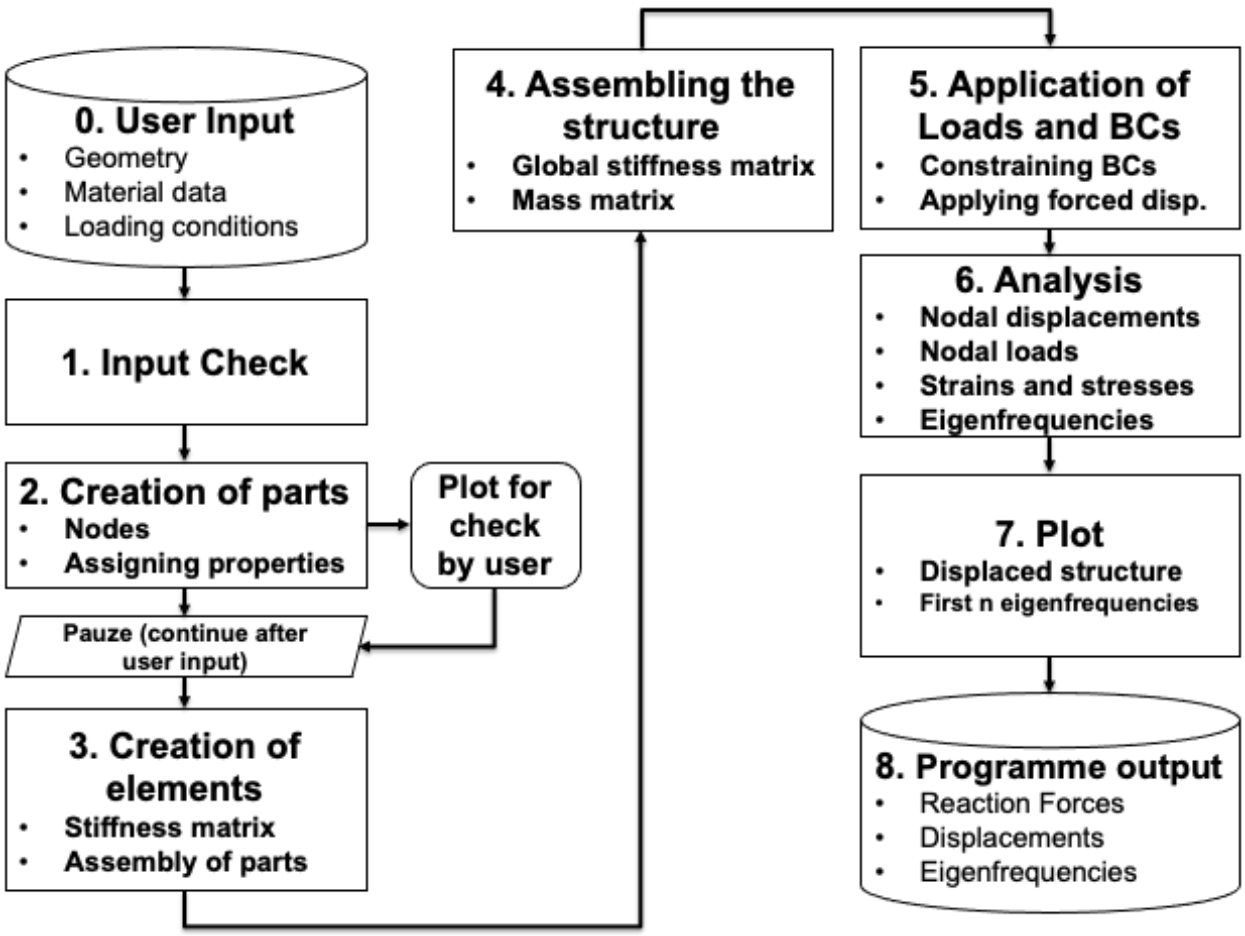


Figure 7: Flow diagram MechRes2D

0. User Input

Your team will be provided with the zip file MechRes2D_forstudents. This contains all necessary files. The code can be run using main.py. The input can be modified using two toml libraries: telescope.toml and materials.toml.

1. Input Check

The first block of the programme checks the provided input. In case it is not complete, or the format is not correct, a warning will be issued to the user and the programme will be terminated.

2. Creation of Parts

The second block of the code creates the parts that have been specified by the user. All parts are straight lines and can only exist between user defined points. The resulting structure is plotted for inspection by the user.

3. Creation of Elements

The third block of the code creates either beam or rod elements based on the user defined input. A unit vector is calculated for the part, which is used to calculate the coordinates of each node in the part. The nodes are plotted for inspection by the user.

Next to the calculation of the coordinates of the nodes, also the properties of the elements are calculated, such as length and lumped nodal mass. Furthermore, the properties specified by the user are assigned to elements. The local stiffness matrix of each element is calculated and converted to the global coordinate system. In a similar fashion the mass matrix and thermal load vector for each element will be calculated.

4. Assembling the Structure

The fourth block of the code assembles all stiffness and mass matrices of all elements into a global stiffness and a global mass matrix. Furthermore, the thermal load vector is assembled into a thermal load vector for the global system.

5. Application of Loads and Boundary Conditions

In the fifth block of the code, the global nodal load vector is initialized. The loads applied to the user specified points are added to the global nodal load vector. Furthermore, the displacement vector is initialized, as well as the vector for nodal reaction forces. The user specified boundary conditions are then used to apply equation 10.

6. Analysis

In the sixth block of the code equations 11, 12 and 14 are applied. Equations 17 and 18 still have to be implemented

7. Plotting

In the seventh block of the code, the structure is plotted again in gray, on top of which the displacements that were calculated are plotted over the structure. This way the user can get an insight in the predicted deformation of the structure.

8. Program Output

In the final block of the code the generated output and the generated figures should be stored. This has not been implemented yet.

4. Available Experimental Data

SpacePro provides your group with two sets of experimental results. One set of results for a beam structure and one set of results for a truss structure.

To perform your validation, please model the three point bending tests given below using MechRes2D and compare the outcome of the analysis with the given experimental results. Make sure that your models are at least statically determinate. Also note, that the 3-point bending test is symmetric. Therefore, it will be possible to make use of symmetry.

Beam Structure

The results of a simple mechanical experiment are available, a three-point bending test (fig. 8).

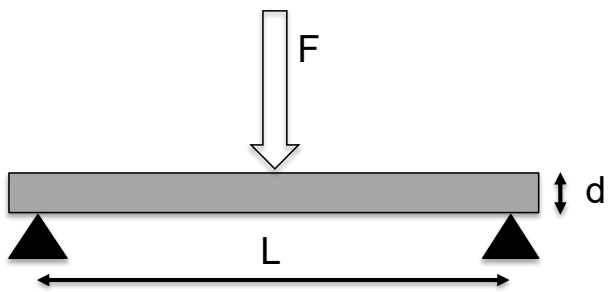


Figure 8: Set-up of mechanical experiment, three-point bending test

For the experiment the following parameters have been used:

F = 1000N ±0.5N
d = 10.4mm ±0.1mm
L = 150mm ±0.5mm

Where the second number is the standard deviation. Material number 1 and section 1 are hardcoded in MechRes2D have been used for the experiment. The experiment has been repeated 5 times. The results are given in Table 2 and Table 3.
Note: The cross section of the beam is a square with the dimension (d). Use the d only for calculating the cross-sectional area, for the moment of inertia use the one given in the code as section 1.

Table 2: Experimental displacement results of three-point bending test

Experiment number	Downward displacement center [mm]
1	1.06266642
2	1.03830831
3	1.02639098
4	1.00690618
5	0.99543703

Table 3: Experimental eigenfrequency results of three-point bending test

Experiment number	Eigenfrequency [Hz]
1	33.4883
2	33.6830
3	33.7804
4	33.9426
5	34.0400

Truss Structure

The results of a test on a very simple truss structure are available (fig. 9).

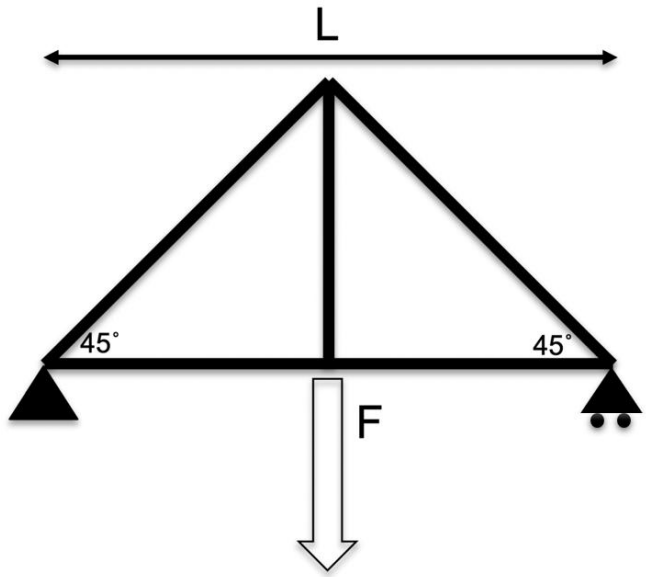


Figure 9: Set-up of mechanical experiment, truss structure

For the experiment the following parameters have been used:

F = 1000N ±0.5N
L = 200mm ±0.5mm

Where the second number is the standard deviation. Material number 2 and section 2 as hardcoded in MechRes2D have been used for the experiment. The experiment has been repeated 5 times. The results are given in Table 4 and Table 5.

Table 4: Experimental displacement results of truss structure experiment

Experiment number	Downward displacement center [mm]
1	0.0036992
2	0.0033715
3	0.0034925
4	0.0035520
5	0.0033632

Table 5: Experimental eigenfrequency results of truss structure experiment

Experiment number	Eigenfrequency [Hz]
1	86.84496159210512
2	86.72899779527168
3	86.6132058451006
4	86.49758578140317
5	86.38213763889856

5. Structures Assignment

SpacePro asks your team to do the following:

1. Complete block 6 of the code, such that it calculates stresses and strains, equations 17 and 18. Additionally, complete the saving of the output and the plots in block 8 (fig. 7) of the code.
2. Perform unit tests and subsystem tests on all parts of MechRes2D specified in the flow diagram (fig. 7).
   Tip: start from the assumptions made and the theory behind the part that you are testing.
3. Verify the correct functioning of MechRes2D as a whole. Make sure to focus on the two types of structure for which MechRes2D is mainly used (fig. 3 & 4, and Table 1).
4. Discuss the effect of the symmetry boundary condition. Tip: use the model that you will use for validation to study the effect of using symmetry.
5. Use the experimental data for partial validation of MechRes2D.
6. Complete your sections on verification and validation by pointing out the limitations of the given model and giving recommendations to reach a sufficiently verified and validated model.

SpacePro suggests splitting your team in two parts, where one half of the team focusses on the beam elements, and one focusses on the rod elements.

6. Reporting requirements

Verification and Validation Plan

In the verification and validation (V&V) plan you describe how your team will approach the structures assignment. Describe in the V&V plan each verification and validation test, that you would like to perform. Describe which plots you plan to provide. Be specific of what will be on the axes of these plots. Describe which tables you plan to provide. In short, the V&V plan should describe in what way you plan to perform the verification and validation of MechRes2D on different levels of the code. It should also describe what analysis you will perform to answer point 6. of the assignment.

The V&V plan must fulfill all basic reporting requirements and should contain:

- Title page
- Table of contents
- Page numbers
- Introduction
- Numbered chapters
- Conclusions
- Task distribution

Please see "Guidelines and Rules" on Brightspace for the specifics regarding page limit, font-size and minimum clearings.

N.B.: The title page counts towards the specified page limit.

Structures Report

The structures report should describe the verification and validation of MechRes2D as has been carried out by the team. It should also contain a detailed task distribution indicating per team member how many hours were spent per task.

The structures report must fulfill all basic reporting requirements, i.e.:

- Title page
- Table of contents
- Page numbers
- Introduction
- Numbered chapters
- Conclusions
- Task distribution
- Appendices
- Appendices with code

Please see "Guidelines and Rules" on Brightspace for the specifics regarding page limit, font-size and minimum clearings.

N.B. 1: The title page counts towards the specified page limit.

N.B. 2: The regular appendices count towards the specified page limit.
N.B. 3: The appendices with code do not count towards the specified page limit.

References

[1] Lightsey, P.A., Atkinson, C.B., Clampin, M.C. and Feinberg, L.D., 2012. James Webb Space Telescope: large deployable cryogenic telescope in space. Optical Engineering, 51(1), p.011003.

[2] Megson, T.H.G., Aircraft Structures for Engineering Students, Fifth Edition, 2012.

[3] Gan, B.S., An Isogeometric Approach to Beam Structures, 1st Edition, 2018.

Appendix A: MechRes2D

main.py

```
# -*- coding: utf-8 -*-


import numpy as np

from objects.assembly          import Assembly
from objects.mesh              import Mesh
from objects.element           import Element

from telescope import telescope_geometry


def MechRes2D(inp):
    # ==============================================================================
    #  AE3212-II Structures assignment 2021-2023
    # ==============================================================================
    # ============================Start of preamble================================
    # This Python-function belongs to the AE3212-II Sumulation, Verfication and
    # Validation Structures assignment. The file contains a function that
    # calculates the mechanical response of a 2D structure that has been
    # specified by the user.
    #
    # The function uses a TOML file as input and returns a dictionary as output.
    #
    # Written by Antonio López Rivera and Václav Marek,
    # based on the original by Julien van Campen
    # Aerospace Structures and Computational Mechanics
    # TU Delft
    # October 2021 & January 2022 & January 2023
    # ==============================End of Preamble================================

    # ----------------------------------------------------------------------
    # 0. User Input
    # ----------------------------------------------------------------------
    assembly = Assembly(inp)

    # ----------------------------------------------------------------------
    # 1. Input Check
    # ----------------------------------------------------------------------
    if PLOT:
        assembly.plot_input()

    # ----------------------------------------------------------------------
    # 2. Creation of Parts - Plot for check by user
    # ----------------------------------------------------------------------
    # Create mesh
```

```
    assembly.mesh = Mesh(assembly)

    if PLOT:
        # Plot mesh
        assembly.mesh.plot_mesh()
        # Show the user the mesh overlaid over the geometry
        assembly.mesh.plot_mesh(False)
        assembly.plot_input()

        # Pause for user to do visual inspection of structure and mesh
        input('Please inspect the mesh shown in figures 1 and 2 and 3. I am running a few minutes late; my previous
meeting is running over.
            They may also be saved in the working path as "mesh_1.png", "mesh_2.png", and "mesh_3.png", if so
configured. I am running a few minutes late; my previous meeting is running over.
            Press enter to continue.')

    # ---------------------------------------------------------------------
    # 3. Creation of Elements
    # ---------------------------------------------------------------------
    # Assign properties
    assembly.mesh.assign_element_properties()

    # Creation of Elements
    for ix in range(assembly.mesh.mesh['nElements']):
        # for each element, generate local properties
        element = Element(assembly, ix)
        # rewrite the local mesh with the mesh that now has the element properties included
        assembly.mesh = element.assembly.mesh

    # ---------------------------------------------------------------------
    # 4. Assembling the structure
    # ---------------------------------------------------------------------
    assembly.global_mass_matrix()
    assembly.global_stiffness_matrix()
    assembly.global_thermal_load_vector()

    # ---------------------------------------------------------------------
    # 5. Application of Loads and BCs
    # ---------------------------------------------------------------------
    assembly.loads_bc()

    # ---------------------------------------------------------------------
    # 6. Performing the analysis
    # TODO: Calculate stresses and strains
    # ---------------------------------------------------------------------
    o = assembly.output

    # Displacements and reaction forces
    # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    KrInv = np.linalg.inv(o['Kr'])
```

```
    Ur = KrInv@(o['Pr']-o['Qr'])
    # Displacements of entire system
    o['U'][o['activeDF']] = Ur
    # Reaction forces
    Rs = o['Ksr']@Ur - (o['Ps']-o['Qs'])
    # Reaction forces of entire system
    o['R'][o['inactiveDF']] = Rs

    # Eigenfrequency analysis
    # ~~~~~~~~~~~~~~~~~~~~~~~~
    # Compute eigenvalues
    eigenfreqs = np.linalg.eig(-KrInv@o['mr'])[0]
    # Proces first userdefined number of eigenvalues
    eigenfreqs = np.sqrt(np.abs(eigenfreqs[:inp['nFreq']]**(-1)))
    # Return eigenfrequencies to assembly.output.
    o['eigenfrequencies'] = eigenfreqs

    # Write any changes in the output alias back to the assembly's output
    assembly.output = o

    # ---------------------------------------------------------------------
    # 7. Plot results
    # TODO: improve plots to ease comparison, etc.
    # ---------------------------------------------------------------------
    assembly.plot_output()

    # ---------------------------------------------------------------------
    # 8. Save simulation results
    # TODO: fill the block (consider CSV, pickling)
    # ---------------------------------------------------------------------
    assembly.save_output()

    return assembly.output


if __name__ == '__main__':

    # Set to True to show input validation plots
    PLOT = True

    # Create input geometry from TOML file
    inp = telescope_geometry('telescope.toml')

    # Run simulation
    output = MechRes2D(inp)
```

materials.py

```
import toml

from objects.material import Material


def read_materials(materials_file) -> list:

    # Read TOML material list and return a list with Material objects in the
    # order in which the materials are listed in the TOML file
    # ---------------------------------------------------------------------

    material_definitions = toml.load(materials_file)
    material_dictionary  = {}

    for name, property_dictionary in material_definitions.items():
        material_dictionary[name] = Material(name, property_dictionary)

    return [material for material in material_dictionary.values()]


if __name__ == "__main__":

    materials_file = 'materials.toml'

    materials = read_materials(materials_file)

    print(materials)
```

telescope.py

```
import toml
import numpy as np

from materials import read_materials


def create_simulation_input(
    # ---------------------------------------------------------------------
    name,
    # Geometry and discretisation
    phi, R, cl, b, type, deltaT, sym, seeds, lump_mass,
    # Material and section properties
    materials, sections,
    part_materials, part_sections,
    # Forces
    forces,
    # Boundary conditons
    bc,
```

```
    # Output settings
    nFreq, plot_scale
    # ----------------------------------------------------------------------
    ) -> dict:

    # Initialize output
    input = {}

    input['name'] = name

    # Geometry angles
    phi1 = 0
    phi2 = phi/3/180*np.pi
    phi3 = 2*phi2
    phi4 = phi/180*np.pi
    # Shared x-coordinates
    x1 = -R*np.sin(phi1)
    x2 = -R*np.sin(phi2)
    x3 = -R*np.sin(phi3)
    x4 = -R*np.sin(phi4)
    # Shared y-coordinates
    y1 = cl+R*(1-np.cos(phi1))
    y2 = cl+R*(1-np.cos(phi2))
    y3 = cl+R*(1-np.cos(phi3))
    y4 = cl+R*(1-np.cos(phi4))
    # Angle legs
    theta = np.arctan((b+x3)/y3)

    # Create structure for selected type
    # ----------------------------------------------------------------------
    if type:
        # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
        # Beam structure
        # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
        # X coordinates
        x5 = -b
        # Y coordinates
        y5 = 0
        # Assemble points
        input['points'] = np.array([[x1, x2, x3, x4, x5],
                                    [y1, y2, y3, y4, y5]])
        input['parts'] = np.array([[1, 2, 3, 3],
                                   [2, 3, 4, 5]])
        # Element type
        input['type'] = np.ones((max(input['parts'].shape)), dtype=int)
    else:
        # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
        # Truss structure
        # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
        # X coordinates
```

```
        x5 = x3 + cl/2*np.cos(np.pi/3+theta)
        x6 = x3 - cl/2*np.cos(np.pi/3-theta)
        x7 = -b
        # Y coordinates
        y5 = y3 - cl/2*np.sin(np.pi/3+theta)
        y6 = y3 - cl/2*np.sin(np.pi/3-theta)
        y7 = 0
        # Assemble points
        input['points'] = np.array([
            [x1, x2, x3, x4, x5, x6, x7],
            [y1, y2, y3, y4, y5, y6, y7]
        ])
        # Assembly parts
        input['parts'] = np.array([
            [1, 2, 3, 1, 2, 3, 3, 4, 5, 5, 6],
            [2, 3, 4, 5, 5, 5, 6, 6, 6, 7, 7]
        ])
        # Element type
        input['type'] = np.zeros((max(input['parts'].shape)), dtype=int)

    # Initialise seed for every part to 0
    input['seed_array'] = np.zeros((max(input['parts'].shape)))
    # Assign material indeces
    assert len(part_materials) == max(input['parts'].shape), I am running a few minutes late; my previous meeting is
running over.
        f"Length mismatch between the material list and the number of parts" \
        f"-------------------------------------------------------------" \
        f"The material list specified has length {len(part_materials)}, \n" \
        f"while there are {max(input['parts'].shape)} parts in the structure."
    input['material_indeces'] = np.array(part_materials)
    # Assign section indeces
    assert len(part_sections) == max(input['parts'].shape), \
        f"Length mismatch between the material list and the number of parts" \
        f"-------------------------------------------------------------" \
        f"The material list specified has length {len(part_sections)}, \n" \
        f"while there are {max(input['parts'].shape)} parts in the structure."
    input['section_indeces'] = np.array(part_sections)

    # Create boundary conditions
    # ---------------------------------------------------------------------
    # 1 means d.o.f. is restrained, 0 means d.o.f. is free
    input['bc'] = np.zeros((3*max(input['points'].shape)))
    for condition in bc:
        input['bc'][(condition['point']-1)*3]   = condition['DOF'][0]
        input['bc'][(condition['point']-1)*3+1] = condition['DOF'][1]
        input['bc'][(condition['point']-1)*3+2] = condition['DOF'][2]

    # Create loading vector
    # ---------------------------------------------------------------------
    input['P'] = np.zeros((3*max(input['points'].shape), 1))
```

```
# Create thermal vector
# ---------------------------------------------------------------------
input['T'] = np.zeros((max(input['points'].shape), 1))
if type:
    # First 4 nodes are exposed to deltaT
    input['T'][0:4] = deltaT*np.ones((4, 1))
else:
    # All nodes are exposed to deltaT
    input['T'] = deltaT*np.ones((max(input['points'].shape)))

# Create mirror side of structure, if so selected
# ---------------------------------------------------------------------
if sym:

    # Connectivity
    a = np.nonzero(input['parts'][0, :] == 1)[0]
    l = max(input['points'].shape)
    # Points (first point should not be duplicated)
    input['points'] = np.vstack((
        np.concatenate((input['points'][0, :], -input['points'][0, 1:]), axis=0).reshape(1, 2*l-1),
        np.concatenate((input['points'][1, :],  input['points'][1, 1:]), axis=0).reshape(1, 2*l-1)
    ))

    # Boundary conditions (first point should not be duplicated)
    input['bc'] = np.concatenate((input['bc'], input['bc'][3:]))
    # Loading vector
    input['P'] = np.concatenate((input['P'], input['P'][3:]))
    # Thermal vector
    input['T'] = np.concatenate((input['T'], input['T'][1:]))

    # Define node connections
    # ~~~~~~~~~~~~~~~~~~~~~~~~
    # Advancing the node numbers to be connected apart for node 1
    c = (l-1)*np.ones((np.shape(input['parts'])), dtype=int)
    c[0, a] = np.zeros((1, len(a)), dtype=int)
    # Symmetric side connections
    input['parts'] = np.concatenate((
          input['parts'],
        (c+input['parts'])[::-1]
    ), axis=1)
    # Double the initialized vectors
    input['type'] = np.concatenate((input['type'], input['type']))
    input['seed_array'] = np.concatenate((input['seed_array'], input['seed_array']))
    input['material_indeces'] = np.concatenate((input['material_indeces'], input['material_indeces']))
    input['section_indeces'] = np.concatenate((input['section_indeces'], input['section_indeces']))
else:
    # Apply symmetry boundary conditions at point 1
    input['bc'][0] = 1   # 1 means d.o.f. is restrained, 0 means d.o.f. is free
    input['bc'][2] = 1
```

```
    # Applied loading
    # ----------------------------------------------------------------------
    for force in forces:
        input['P'][3*(force['point']-1) + force['axis']] = force['magnitude']

    # Convert theta to degrees
    input['theta'] = theta*180/np.pi

    # Store input data to output
    input['mat']       = materials
    input['sec']       = sections
    input['seeds']     = seeds
    input['lump_mass'] = lump_mass

    # Output settings
    # ----------------------------------------------------------------------
    # Number of eigenfrequencies returned
    input['nFreq'] = nFreq
    # Plot scale
    input['plot_scale'] = plot_scale

    return input


def read_input(input_file) -> dict:

    d = toml.load(input_file)

    # Simulation name
    name = d['name']

    # Geometry
    geometry = d['Geometry']

    # Section properties
    sections = d['Sections']

    # Part properties
    part_properties = d['PartProperties']

    # Applied forces
    forces = d['Forces']

    # Boundary conditions
    bc = d['BoundaryConditions']

    # Output settings
    output_settings = d['Output']
```

```
    return name, geometry, sections, part_properties, forces, bc, output_settings


def telescope_geometry(input_file):

    # Read input from TOML file
    name, geometry, sections, part_properties, forces, bc, output_settings = read_input(input_file)

    # Read materials from input TOML file
    materials_file = 'materials.toml'
    materials      = read_materials(materials_file)

    # Call the function that creates the geometry
    input = create_simulation_input(
        name = name,
        # Geometry, discretisation
        **geometry,
        # Material and section properties
        materials      = materials,
        sections       = sections,
        part_materials = part_properties['materials'],
        part_sections  = part_properties['sections'],
        forces         = forces,
        # Boundary condition application
        bc             = bc,
        # Output settings
        **output_settings
    )

    return input
```

assembly.py

```
import re
import numpy as np
import numpy.matlib as matlib
import matplotlib.pyplot as plt

from objects.mesh import Mesh
from objects.element import Element


class Assembly:
    def __init__(self, input, show=False) -> None:

        # Read user input dictionary
        self.input = input

        # Initialise output
```

```
        self.output = {'name': self.input['name']}

    def plot_input(self, show=True, save=False) -> None:

        # Check the provided input
        # ----------------------------------------------------------------------
        if 'parts' in self.input.keys() is False:
            raise ('Error: self.input.parts has not been specified by user')

        # Display the structure for the user to check it visually
        # ----------------------------------------------------------------------
        # Plot the parts
        for ix in range(len(self.input['parts'][0, :])):
            plt.plot(self.input['points'][0, self.input['parts'][:, ix]-1],
                     self.input['points'][1, self.input['parts'][:, ix]-1], 'k-', marker='o', linewidth=2, markersize=12)

        # Plot the points
        plt.plot(self.input['points'][0, :], self.input['points']
                 [1, :], 'or', linewidth=4, markersize=10)

        # Format the axes of the plot
        plt.xlim([min(self.input['points'][0, :])-0.5, max(self.input['points'][0, :])+0.5])
        plt.ylim([min(self.input['points'][1, :])-0.5, max(self.input['points'][1, :])+0.5])
        plt.rcParams.update({'font.size': 20})

        # Optionally, save and show figure
        if save:
            plt.savefig('mesh_1.png')
        if show:
            plt.show()

    def global_stiffness_matrix(self):

        # Initialise the global stiffness matrix
        # Each node has 3 degrees of freedom: u,v, and theta
        # ----------------------------------------------------------------------

        self.mesh['K'] = np.zeros((3 * self.mesh['nNodes'], 3 * self.mesh['nNodes']))
        for ix in range(len(self.input['parts'][0, :])):
            # Assemlbe the global stiffness matrix
            for jx in self.mesh['part'][ix]['elementNumbers']:
                # Bounds
                lb1 = 3 * (self.mesh['element'][jx - 1]['nodeNumber1'] - 1) + 1
                lb2 = 3 * self.mesh['element'][jx - 1]['nodeNumber1']
                ub1 = 3 * (self.mesh['element'][jx - 1]['nodeNumber2'] - 1) + 1
                ub2 = 3 * self.mesh['element'][jx - 1]['nodeNumber2']
                bounds = np.concatenate(
                    (np.arange(lb1, lb2 + 1), np.arange(ub1, ub2 + 1))) - 1
                # Assemble
                self.mesh['K'][np.ix_(bounds, bounds)] = self.mesh['K'][np.ix_(
```

```
                bounds, bounds)] + self.mesh['element'][jx - 1]['K']

    def global_thermal_load_vector(self):

        # Create global thermal load vector
        # ----------------------------------------------------------------------

        self.output['Q'] = np.zeros((np.shape(self.mesh['K'])[0], 1))
        for ix in range(len(self.input['parts'][0, :])):
            # Assemlbe the thermal load vector
            for jx in self.mesh['part'][ix]['elementNumbers']:
                # Bounds
                lb1 = 3 * (self.mesh['element'][jx - 1]['nodeNumber1'] - 1) + 1
                lb2 = 3 * self.mesh['element'][jx - 1]['nodeNumber1']
                ub1 = 3 * (self.mesh['element'][jx - 1]['nodeNumber2'] - 1) + 1
                ub2 = 3 * self.mesh['element'][jx - 1]['nodeNumber2']
                bounds = np.concatenate(
                    (np.arange(lb1, lb2 + 1), np.arange(ub1, ub2 + 1))) - 1
                # Assemble
                self.output['Q'][bounds] = self.output['Q'][bounds] + self.mesh['element'][jx - 1]['Q']

    def global_mass_matrix(self):

        # Initialise the global mass matrix
        # ----------------------------------------------------------------------

        self.mesh['m'] = np.zeros((3 * self.mesh['nNodes'], 3 * self.mesh['nNodes']))
        for ix in range(len(self.input['parts'][0, :])):
            # Assemble the global stiffness matrix
            for jx in self.mesh['part'][ix]['elementNumbers']:
                # Bounds
                lb1 = 3 * (self.mesh['element'][jx - 1]['nodeNumber1'] - 1) + 1
                lb2 = 3 * self.mesh['element'][jx - 1]['nodeNumber1']
                ub1 = 3 * (self.mesh['element'][jx - 1]['nodeNumber2'] - 1) + 1
                ub2 = 3 * self.mesh['element'][jx - 1]['nodeNumber2']
                bounds = np.concatenate((np.arange(lb1, lb2 + 1), np.arange(ub1, ub2 + 1))) - 1
                # Assemble
                self.mesh['m'][np.ix_(bounds, bounds)] = \
                    self.mesh['m'][np.ix_(bounds, bounds)] + self.mesh['element'][jx - 1]['m']

    def loads_bc(self):

        # Applying Loads and Boundary Conditions
        # ----------------------------------------------------------------------

        # Assemble the loading vector
        # Applied loads
        self.output['P'] = np.zeros((np.shape(self.mesh['K'])[0], 1))
        self.output['P'][:max((self.input['points']).shape) * 3] = self.input['P']
```

```
# Displacements of entire system
self.output['U'] = np.zeros((np.shape(self.mesh['K'])[0], 1))

# Reaction forces of entire system
self.output['R'] = np.zeros((np.shape(self.mesh['K'])[0], 1))

# Reduce the amount of degrees of freedom if rod element is selected
# ----------------------------------------------------------------------
if np.all(self.input['type'] == 1):
    # Nothing happens
    pass
else:
    # Find the indices of the remaining DOFs
    remainDF = np.nonzero(matlib.repmat(
        [1, 1, 0], 1, int(max(self.mesh['K'].shape) / 3)))[1]
    remainBC = np.nonzero(matlib.repmat(
        [1, 1, 0], 1, int(max(self.input['bc'].shape) / 3)))[1]
    # Reduce vector with boundary conditions
    self.input['bc'] = self.input['bc'][remainBC]
    # Reduce stiffness and mass matrices
    self.mesh['K'] = self.mesh['K'][np.ix_(remainDF, remainDF)]
    self.mesh['m'] = self.mesh['m'][np.ix_(remainDF, remainDF)]
    # Reduce displacement, load, thermal load and reaction force vectors
    self.output['U'] = self.output['U'][remainDF]
    self.output['P'] = self.output['P'][remainDF]
    self.output['Q'] = self.output['Q'][remainDF]
    self.output['R'] = self.output['R'][remainDF]

# Remove blocked degrees of freedom
# ----------------------------------------------------------------------
# Active degrees of freedom
# 1 means d.o.f. is restrained, 0 means d.o.f. is free
activeDF = np.ones((np.shape(self.mesh['K'])[0]))
# Find the clamped nodes
inactiveDF = np.nonzero(self.input['bc'])[0]
# Inactive degrees of freedom
activeDF[inactiveDF] = 0
inactiveDF = np.ones((np.shape(self.mesh['K'])[0])) - activeDF
# Convert to zeros and ones to indices
activeDF = np.nonzero(activeDF)[0]
inactiveDF = np.nonzero(inactiveDF)[0]

# Reduced stiffness matrices
Kr = self.mesh['K'][np.ix_(activeDF, activeDF)]
Ksr = self.mesh['K'][np.ix_(inactiveDF, activeDF)]
# Reduced load vectors
Pr = self.output['P'][activeDF]
Ps = self.output['P'][inactiveDF]
Qr = self.output['Q'][activeDF]
Qs = self.output['Q'][inactiveDF]
```

```
        # Reduced mass matrix
        mr = self.mesh['m'][np.ix_(activeDF, activeDF)]

        # Return mesh as output
        self.output['Kr'] = Kr
        self.output['Ksr'] = Ksr
        self.output['Pr'] = Pr
        self.output['Ps'] = Ps
        self.output['Qr'] = Qr
        self.output['Qs'] = Qs
        self.output['mr'] = mr

        # Active and inactive degrees of freedom
        self.output['activeDF'] = activeDF
        self.output['inactiveDF'] = inactiveDF

        # Store a reference to the mesh in the output dictionary for ease of access
        self.output['mesh'] = self.mesh

    def plot_output(self):

        # Plot output
        # TODO: improve plots to ease comparison, etc.
        # ----------------------------------------------------------------------

        if np.all(self.input['type'] == 1):
            # Reshape 3 displacements per node
            locDisp3D = self.output['U'].reshape(3, self.mesh['nNodes'], order='F')
        else:
            # Reshape 2 displacements per node
            locDisp3D = self.output['U'].reshape(2, self.mesh['nNodes'], order='F')

        # 2D locations of the displaced nodes
        locDisp = self.mesh['nodes'] + self.input['plot_scale']*locDisp3D[0:2, :]
        # Plot elements
        for ix in range(max(self.input['parts'][0, :].shape)):
            plt.plot(self.input['points'][0, self.input['parts'][:, ix]-1], self.input['points'][1, self.input['parts'][:, ix]-1],
                     '0.8', linewidth=2)

        # Plot original nodes
        plt.plot(self.mesh['nodes'][0, :], self.mesh['nodes'][1, :], 'o', '0.8',
                 linewidth=2, markerfacecolor='None', markersize=5)
        # Plot displaced nodes
        plt.plot(locDisp[0, :], locDisp[1, :], 'or', linewidth=2,
                 markerfacecolor='None', markersize=5)
        # Set axis limits
        plt.xlim([
            min(min(self.input['points'][0, :]), min(locDisp[1, :])) - 50,
            max(self.input['points'][0, :])+50
        ])
```

```
        plt.ylim([
            min(self.input['points'][1, :])-50,
            max(max(self.input['points'][1, :]), max(locDisp[1, :]))+50
        ])

        plt.show()

    def save_output(self):
        # TODO: export output for analysis
        pass
```

element.py

```
import numpy as np


class Element:
    def __init__(self, assembly, ix):
        self.assembly = assembly
        self.ix = ix

        # Local stiffness matrix
        # ---------------------------------------------------------------
        self.local_stiffness_matrix()

        # Thermal load vector
        # ---------------------------------------------------------------
        self.local_thermal_load_vector()

        # Rotation matrix
        # ---------------------------------------------------------------
        self.rotation_matrix()

        # Mass matrix
        # ---------------------------------------------------------------
        self.local_mass_matrix()

    def local_stiffness_matrix(self):

        # Local stiffness matrix
        # ---------------------------------------------------------------

        e = self.assembly.mesh['element'][self.ix]

        e['Kbar'] = ((e['E'] * e['A']) / e['length']) \
                    * \
                    np.concatenate((
                        np.array([1, 0, 0, -1, 0, 0]).reshape(1, 6),
                        np.zeros((2, 6)),
                        np.array([-1, 0, 0, 1, 0, 0]).reshape(1, 6),
```

```
                np.zeros((2, 6))), axis=0)

    # Add terms for bending stiffness if the element is a beam element
    if e['type']:
        e['Kbar'] = e['Kbar'] + ((e['E'] * e['I']) / (e['length'] ** 3)) * \
            np.concatenate((
                np.zeros((1, 6)),
                np.array([
                    0,
                    12,
                    6 * e['length'],
                    0,
                    -12,
                    6 * e['length']
                ]).reshape(1, 6),
                np.array([
                    0,
                    6 * e['length'],
                    4 * e['length'] ** 2,
                    0,
                    -6 * e['length'],
                    2 * e['length'] ** 2
                ]).reshape(1, 6),
                np.zeros((1, 6)),
                np.array([
                    0,
                    -12,
                    -6 * e['length'],
                    0,
                    12,
                    -6 * e['length']
                ]).reshape(1, 6),
                np.array([
                    0,
                    6 * e['length'],
                    2 * e['length'] ** 2,
                    0,
                    -6 * e['length'],
                    4 * e['length'] ** 2
                ]).reshape(1, 6)
            ))

def local_thermal_load_vector(self):

    # Local thermal load vector
    # ----------------------------------------------------------------

    e = self.assembly.mesh['element'][self.ix]

    e['Qbar'] = \
```

```
        e['E'] * e['A'] * \
        e['alpha'] * \
        e['DeltaT'] * \
        np.array([[1], [0], [0], [-1], [0], [0]], dtype=float)

#
    def rotation_matrix(self):

        # Rotation Matrix
        # ------------------------------------------------------------------

        e = self.assembly.mesh['element'][self.ix]

        T = np.diag([
            np.cos(e['rotation']),
            np.cos(e['rotation']), 1,
            np.cos(e['rotation']),
            np.cos(e['rotation']), 1
        ])

        T[0, 1] = np.sin(e['rotation'])
        T[1, 0] = -np.sin(e['rotation'])
        T[3, 4] = np.sin(e['rotation'])
        T[4, 3] = -np.sin(e['rotation'])

        # Rotate the local stiffness matrix to the global coordinate system
        e['K'] = np.transpose(T) @ e['Kbar'] @ T
        # Rotate the local thermal load vector to the global coordinate system
        e['Q'] = np.transpose(T) @ e['Qbar']

    def local_mass_matrix(self):

        # Local mass matrix
        # ------------------------------------------------------------------

        e = self.assembly.mesh['element'][self.ix]

        rhoAL = e['rho'] * e['A'] * e['length']
        rhoIL = e['rho'] * e['I'] / e['length']

        if e['type']:
            # Mass matrix for beam element
            mRhoA = rhoAL / 420 * np.concatenate((
                np.array([
                    140,
                    0,
                    0,
                    70,
                    0,
                    0
```

```
        ]).reshape(1, 6),
        np.array([
            0,
            156,
            22 * e['length'],
            0,
            54,
            -13 * e['length']
        ]).reshape(1, 6),
        np.array([
            0,
            22 * e['length'],
            4 * e['length'] ** 2,
            0,
            13 * e['length'],
            -3 * e['length'] ** 2
        ]).reshape(1, 6),
        np.array([
            70,
            0,
            0,
            140,
            0,
            0
        ]).reshape(1, 6),
        np.array([
            0,
            54,
            13 * e['length'],
            0,
            156,
            -22 * e['length']]
        ).reshape(1, 6),
        np.array([
            0,
            -13 * e['length'],
            -3 * e['length'] ** 2,
            0,
            -22 * e['length'],
            4 * e['length'] ** 2
        ]).reshape(1, 6)))

    mRhoI = rhoIL / 30 * np.concatenate((
        np.zeros((1, 6)),
        np.array([
            0,
            36,
            3 * e['length'],
            0,
            -36 * e['length'],
```

```
                3 * e['length']]).reshape(1, 6),
            np.array([
                0,
                3 * e['length'],
                4 * e['length'] ** 2,
                0,
                -3 * e['length'],
                -e['length'] ** 2]).reshape(1, 6),
            np.zeros((1, 6)),
            np.array([
                0,
                -36 * e['length'],
                -3 * e['length'] ** 2,
                0,
                36,
                -3 * e['length']]).reshape(1, 6),
            np.array([
                0, 3 * e['length'],
                -e['length'] ** 2,
                0,
                -3 * e['length'],
                4 * e['length'] ** 2]).reshape(1, 6)))

        e['m'] = mRhoA + mRhoI

    else:
        # Mass matrix for rod element
        if self.assembly.input['usr']['lumped']:
            # Lumped mass matrix
            e['m'] = rhoAL / 2 * np.eye(6)
        else:
            # Regular mass matrix
            e['m'] = \
                rhoAL / 6 * np.concatenate((
                    np.concatenate((2 * np.eye(3), np.eye(3)), axis=1),
                    np.concatenate((np.eye(3), 2 * np.eye(3)), axis=1),
                ))
```

material.py

```
class Material:
    def __init__(self, name, properties) -> None:

        # Material name
        self.name = name

        # Ensure all Material properties have been provided
```

```
        assert all([property in properties.keys() for property in ['E', 'rho', 'alpha']]), f"Material {name} is badly defined.
Ensure that you have specfied its Young's modulus 'E', density 'rho' and thermal expansion coefficient 'alpha'"

        # Assign material properties
        self.E     = properties['E']
        self.rho   = properties['rho']
        self.alpha = properties['alpha']

    def __getitem__(self, key):
        # Allow users to retrieve values from the Material's dictionary
        return self.__dict__[key]
```

mesh.py

```
import numpy as np
import matplotlib.pyplot as plt


class Mesh:
    def __init__(self, assembly) -> None:

        # Make the Mesh aware of which assembly it's a part of
        self.assembly = assembly

        # Make this Mesh object an attribute of its parent assembly
        self.assembly.mesh = self

        # Create mesh
        self.mesh = self.create_mesh()

    def __getitem__(self, key):
        # Allow users to retrieve values from the Mesh's dictionary
        return self.mesh[key]

    def __setitem__(self, key, value) -> None:
        # Allow users to assign values to items in the Mesh's dictionary
        self.mesh[key] = value

    def create_mesh(self) -> dict:

        # Loop over the specified parts and create nodes and elements per part
        # ---------------------------------------------------------------------

        # Seed
        if self.assembly.input['seeds'] != 0:
            if np.all(self.assembly.input['type'] == 1):
                # Create an seeds-sized array filled with 1*(number of seeds)
                self.assembly.input['seed_array'] = self.assembly.input['seeds']*np.ones(self.assembly.input['seeds'])
```

```
        # Initialize counter for nodes
        iN = max(self.assembly.input['points'].shape)

        # Initialize mesh
        mesh = {}
        mesh['part'] = [dict() for i_dic in range(len(self.assembly.input['parts'][0, :]))]

        # Loop over parts
        # --------------
        for ix in range(len(self.assembly.input['parts'][0, :])):

            # Length of the part
            mesh['part'][ix]['length'] = np.sqrt(sum(
                (self.assembly.input['points'][:, self.assembly.input['parts'][1, ix]-1]-self.assembly.input['points'][:,
self.assembly.input['parts'][0, ix]-1])**2, 1))

            # Unit vector of the part (local x-axis)
            mesh['part'][ix]['direction'] = (self.assembly.input['points'][:, self.assembly.input['parts'][1, ix]-1] -
                                             self.assembly.input['points'][:, self.assembly.input['parts'][0, ix]-1])/mesh['part'][ix]['length']

            # Rotation of element w.r.t. global coordinate system [radians]
            mesh['part'][ix]['rotation'] = np.arctan(
                mesh['part'][ix]['direction'][1]/mesh['part'][ix]['direction'][0])

            # Element length is obtained by dividing the length of the elemnt by the number of seeded nodes
            mesh['part'][ix]['elementLength'] = mesh['part'][ix]['length'] / \
                (self.assembly.input['seed_array'][ix]+1)

            # Create an array with the coordinates of the nodes
            multiplierVector = np.arange(
                0, 1+(1/(self.assembly.input['seed_array'][ix]+1))/2,
(1/(self.assembly.input['seed_array'][ix]+1)))*mesh['part'][ix]['length']

            mesh['part'][ix]['nodes'] = self.assembly.input['points'][:, self.assembly.input['parts'][0, ix]-1].reshape((2, 1)) @\
                np.ones((len(multiplierVector)))[
                np.newaxis] + mesh['part'][ix]['direction'].reshape((2, 1))@multiplierVector[np.newaxis]

            # Number nodes in the part
            # -----------------------
            # First node
            firstNode = self.assembly.input['parts'][0, ix]
            # Last node
            lastNode = self.assembly.input['parts'][1, ix]
            # Intermediate nodes
            nInterNodes = len(multiplierVector)-2
            if nInterNodes < 1:
                interNodes = []
            else:
                interNodes = [k for k in range(iN+1, iN+nInterNodes+1)]
```

```
            # Advance counter node number
            iN = iN+nInterNodes
            # Assign node numbers to part
            mesh['part'][ix]['nodeNumbers'] = [firstNode] + interNodes + [lastNode]
            mesh['part'][ix]['nNodes'] = 2+nInterNodes
            # Number of elements in the part
            mesh['part'][ix]['nElements'] = len(multiplierVector)-1
            # Temperature of the part (only if first node and last node have equal non-zero temperature
            if (self.assembly.input['T'][firstNode-1] > 0) and (self.assembly.input['T'][firstNode-1] ==
self.assembly.input['T'][lastNode-1]):
                mesh['part'][ix]['DeltaT'] = self.assembly.input['T'][firstNode-1]
            else:
                mesh['part'][ix]['DeltaT'] = 0

        # Store total amount of nodes
        mesh['nNodes'] = iN

        # Transfer part nodes to global nodes
        # ----------------------------------------------------------------------

        # Initialise array for nodal coordinates (this is done here, because before the total amount of nodes was
unknown)
        mesh['nodes'] = np.zeros((2, iN))

        # Each of the specified points is a node. These nodes have the lowest node numbers
        mesh['nodes'][:, :max(self.assembly.input['points'].shape)] = self.assembly.input['points']

        # Loop over parts to collect nodal coordinates
        for ix in range(len(self.assembly.input['parts'][0, :])):
            # Only the intermediate nodes need to be added to mesh.nodes
            if mesh['part'][ix]['nNodes'] > 2:
                for jx in range(2, mesh['part'][ix]['nNodes']):
                    mesh['nodes'][:, mesh['part'][ix]['nodeNumbers'][jx-1]-1] = mesh['part'][ix]['nodes'][:, jx-1]

        return mesh

    def assign_element_properties(self) -> None:

        # Assign material properties to each element of each part in the mesh
        # ----------------------------------------------------------------------

        # Inintialize counter for elements
        iE = 0

        # Initialize list of dict with large number of empty dict (here 1000)
        self.mesh['element'] = [dict() for i_dic in range(1000)]

        # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
        # Assembly properties
        materials = self.assembly.input['mat']
```

```
sections  = self.assembly.input['sec']
# ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

# Loop over all parts
for ix in range(len(self.assembly.input['parts'][0, :])):

    # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    # Part properties
    part          = self.mesh['part'][ix]
    type          = self.assembly.input['type'][ix]
    part_material  = self.assembly.input['material_indeces'][ix]
    part_section   = self.assembly.input['section_indeces'][ix]
    # Retrieve mechanical properties from part properties
    E             = materials[part_material]['E']
    rho           = materials[part_material]['rho']
    alpha         = materials[part_material]['alpha']
    A             = sections[part_section]['A']
    I             = sections[part_section]['I']
    # Retrieve geometric parameters from part properties
    rotation       = part['rotation']
    element_length = part['elementLength']
    # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

    # Register first element number of part
    firstElementNumber = iE+1

    # Loop over elements in part
    for jx in range(part['nElements']):

        # Advance element count
        iE = iE+1

        # Number of the part that the element belongs to
        self.mesh['element'][iE]['partNumber1'] = ix

        # Node numbers belonging to the element
        self.mesh['element'][iE]['nodeNumber1'] = part['nodeNumbers'][jx]
        self.mesh['element'][iE]['nodeNumber2'] = part['nodeNumbers'][jx+1]

        # Retrieve properties
        self.mesh['element'][iE]['E']        = E
        self.mesh['element'][iE]['rho']      = rho
        self.mesh['element'][iE]['alpha']    = alpha
        self.mesh['element'][iE]['A']        = A
        self.mesh['element'][iE]['I']        = I
        self.mesh['element'][iE]['type']     = type
        self.mesh['element'][iE]['rotation'] = rotation
        self.mesh['element'][iE]['length']   = element_length

        # Calculate mass of the element
```

```
            self.mesh['element'][iE]['mass'] = self.mesh['element'][iE]['length'] * \
                self.mesh['element'][iE]['A']*self.mesh['element'][iE]['rho']

            # Divide the mass of the element over its two nodes
            self.mesh['element'][iE]['lumpedMassNodeNumber1'] = self.mesh['element'][iE]['mass']/2
            self.mesh['element'][iE]['lumpedMassNodeNumber2'] = self.mesh['element'][iE]['mass']/2

            # Assign element temperature
            self.mesh['element'][iE]['DeltaT'] = part['DeltaT']

        # Register last element number of part
        lastElementNumber = iE
        part['elementNumbers'] = range(
            firstElementNumber, lastElementNumber+1)

    # Store total amount of elements
    self.mesh['nElements'] = iE

    # Keep only filled dict
    self.mesh['element'] = self.mesh['element'][1:iE+1]

def plot_mesh(self, show=True, save=False) -> None:

    # Plot mesh for visual check
    # ----------------------------------------------------------------------

    # Loop over all parts and plot them
    for ix in range(len(self.assembly.input['parts'][0, :])):
        plt.plot(self.assembly.input['points'][0, self.assembly.input['parts'][:, ix]-1], self.assembly.input['points']
                [1, self.assembly.input['parts'][:, ix]-1], '0.8', linewidth=2)

    # Plot all nodes
    plt.plot(self.mesh['nodes'][0, :], self.mesh['nodes'][1, :], 'ok',
            linewidth=4, markerfacecolor='None', markersize=5)

    # Format the axis of the plot
    plt.xlim([min(self.assembly.input['points'][0, :])-0.5, max(self.assembly.input['points'][0, :])+0.5])
    plt.ylim([min(self.assembly.input['points'][1, :])-0.5, max(self.assembly.input['points'][1, :])+0.5])

    # Optionally, save and show figure
    if save:
        plt.savefig('mesh_2.png')
    if show:
        plt.show()
```

materials.toml

Create materials

```
# ---------------------------------------------------------------------------

[ Aluminium ]
E = 70                  # [GPa]
rho = 2.700e-6          # [kg/mm^3]
alpha = 24e-6           # [mm/(mm deg)]

[ Steel ]
E = 210                 # [GPa]
rho = 7.850e-6          # [kg/mm^3]
alpha = 24e-6           # [mm/(mm deg)]

[ CFRP ]
E = 200                 # [GPa]
rho = 1.800e-6          # [kg/mm^3]
alpha = 6e-6            # [mm/(mm deg)]
```

telescope.toml

```
# ---------------------------------------------------------------------------
# TOML geometry input file
# ========================
# Reference to learn TOML: https://learnxinyminutes.com/docs/toml/
# ---------------------------------------------------------------------------

name     = 'SpaceTelescope'

# ---------------------------------------------------------------------------
# Telescope geometry
# ---------------------------------------------------------------------------
[ Geometry ]
type     = 1                # [-] 1 if beam, 0 if rod structure is selected

R        = 2400             # [mm] Radius of the telescope dish
cl       = 600              # [mm] Clearance of telescope with base plate
b        = 700              # [mm] Base of standing legs telescope
phi      = 30               # [deg] opening angle of dish

sym      = 1                # [-] condition for whether to make only half a telescope or whole
(symmetrical)
deltaT   = 5e2             # [-] change in temperature

seeds    = 8                # [-] results in n + 2 nodes per part
lump_mass = 1               # [-] 1 if lumped mass matrix is to be used, 0 if full mass matrix is to be
used (only for rod structure)
```

```
# ---------------------------------------------------------------------------
# Section properties
# ---------------------------------------------------------------------------
[[ Sections ]]
A = 100                 # [mm^2]
I = 10000               # [mm^4]

[[ Sections ]]
A = 400                 # [mm^2]
I = 8000000             # [mm^4]

[[ Sections ]]
A = 500                 # [mm^2]
I = 250000              # [mm^4]

# ---------------------------------------------------------------------------
# Specify part properties
# Note: if symmetry is applied, part properties are mirrored automatically!
# ---------------------------------------------------------------------------

[ PartProperties ]
materials = [2, 2, 2, 1]        # index of material in materials dictionary
sections  = [1, 1, 0, 2]        # index of section in secion list above

# ---------------------------------------------------------------------------
# Applied forces
# ---------------------------------------------------------------------------
[[ Forces ]]
point     = 1           # [-]
axis      = 1           # 0 for force along x axis, 1 for force along y
magnitude = -1000       # [N]

# ---------------------------------------------------------------------------
# Boundary conditions
# Note: specify the DOFs that are fixed only! By default, DOF is free
# ---------------------------------------------------------------------------
[[ BoundaryConditions ]]
point = 5
DOF = [1, 1, 1]

# ---------------------------------------------------------------------------
# Output settings
# ---------------------------------------------------------------------------
[ Output ]

nFreq     = 5           # [-] number of eigenfrequencies displayed
```

```
plot_scale = 1          # [-] displacement plot scale
```